

HelpLK AI — Team Briefing

Agentic Citizen Services Copilot for Sri Lanka · “From confusion to completion.”

Internal preparation document for the evaluation meeting · July 2026 · Companion to PRESENT.md (the full demo playbook)

The one-liner every team member should know cold:

“ChatGPT answers questions. HelpLK AI *executes government procedures* — it plans the correct steps in the correct order, checks your eligibility, tracks your documents, and remembers your progress until you finish.”

1 What the Platform Does (Basic Functionalities)

Functionality	What actually happens
Natural-language goal intake	Citizen types their situation (“I lost my NIC and need a passport”) — no department, form name, or office required. Sinhala / Tamil / English selector on the goal page.
Agentic plan generation	A team of 8 specialized AI agents runs as a LangGraph state machine. The processing screen animates from <i>live server-sent events emitted by real agent execution</i> — not a fake timer.
Eligibility screening (human-in-the-loop)	Before the plan is built, the run <i>pauses</i> and asks clarifying questions (citizenship, age, relationships...). For curated services the questions come from deterministic rules; for novel goals they are generated on the fly. Ineligible citizens get a locked plan <i>plus an alternative route suggestion</i> .
Dependency-aware workflow	Prerequisites are computed from a rules dataset, never guessed by the LLM. A lost NIC automatically inserts the duplicate-NIC procedure <i>before</i> passport steps and locks the passport steps until it is done.
Step tracking	Citizens mark steps done (or undo). Progress %, the “next best action”, and lock states recompute server-side; completing all prerequisite steps visibly unlocks dependent ones.
Document checklist & verification	Each case gets a checklist of only the documents <i>that case</i> requires. Uploads are verified by Gemini Vision (Tesseract OCR fallback) — type detection, issue list (blur, missing signature), accepted / incomplete / rejected verdicts.
Citizen dashboard & case memory	All cases with live progress. State survives across sessions in two layers: durable Postgres tables + a LangGraph checkpointer, so “continue my process days later” genuinely works.
Explainability & audit	Every step links to its official government source. Every agent decision is logged with reason, source, and confidence (the agent_logs audit trail).
Custom procedures	Goals outside the curated dataset (citizenship for a foreign spouse, property transfer) get a coherent LLM-generated plan — still passed through the same eligibility and dependency engines.

2 The Agents

Agent	Responsibility	Intelligence
Planner	Understands the goal → detected services + intent facts (what the citizen has / lost / needs). Generates a full custom plan when no known service fits.	Groq Llama 3.3 70B (JSON mode), keyword fallback
Knowledge (RAG)	Collects requirements per service and attaches citations from official gov.lk sources.	Rules JSON + pgvector retrieval
Dependency	Builds the prerequisite graph, locks blocked services, injects prerequisite procedures (lost NIC → duplicate-NIC first).	Deterministic rules — no LLM
Eligibility	Asks minimal clarifying questions, evaluates eligibility, suggests alternative paths when blocked.	Rules + Groq (questions/verdicts for custom goals)
Checklist	Composes the ordered step list, computes progress, marks the next best action, persists steps and the required-documents list.	Deterministic composition
Document Verify	Analyzes uploaded document images: type, visible fields, issues.	Gemini 2.5 Flash vision → Tesseract fallback
Form Assistant	Explains and validates government form fields on demand.	Groq
Reminder	Computes due / expiry / idle nudges from case state.	Deterministic
Audit & Trust	Cross-cutting: every agent writes decision + reason + source + confidence to the audit log.	—

How the agents communicate

Agents never message each other directly. Each is a node in one **LangGraph state machine** reading and writing a shared typed state (GraphState); LangGraph merges each node's partial update and routes along fixed + conditional edges:

```
START → Planner → Knowledge → Dependency → Eligibility
    ↳ needs info?  || PAUSE (ask citizen) → answers injected → Eligibility re-runs
    → Checklist → || PAUSE (await document upload) → Document Verify → Form → Reminder → END
```

The two **||** pauses are **human-in-the-loop interrupts**: the graph checkpoints itself to Postgres (thread = case) and resumes later from exactly that point — this is the technical heart of the “stateful, long-running case” story. Node progress streams to the browser over **SSE**, driving the processing animation.

3 Hybrid RAG — Our Anti-Hallucination Design

Two retrieval layers with different jobs. **This split is our strongest technical talking point:**

- **Deterministic rules layer** — one JSON file per service (requirements, dependencies, eligibility predicates, steps, official source URL). Decides *structure*: ordering, locking, eligibility. Exact lookup — the LLM cannot hallucinate a wrong procedure order.
- **Vector RAG layer** — real gov.lk pages (Immigration, DRP, RMV, Registrar General) chunked and embedded (Gemini gemini-embedding-001, 768-dim) into Postgres pgvector; cosine search supplies *citations and narrative context*.

Quote for judges: “Rules decide structure, LLMs decide language, retrieval provides evidence.”

4 Tech Stack (all free tier)

Layer	Technology
Frontend	Next.js 16 (App Router) · Tailwind v4 · Framer Motion (animation driven by real SSE events)
Backend	FastAPI (Python) · LangGraph agent graph with Postgres checkpointer
Data	Postgres + pgvector (Neon) — domain tables, agent checkpoints, and embeddings in one data plane
LLMs	Groq Llama 3.3 70B (reasoning, JSON mode) · Gemini 2.5 Flash (vision, translation) · gemini-embedding-001
Resilience	Fallback ladders everywhere: Groq→keyword rules · Gemini embeddings→local sentence-transformers · Gemini Vision→Tesseract OCR · checkpointer→graceful no-persistence mode
Auth	Signed HS256 JWT verified by FastAPI; every query scoped by user id (demo-mode identity for the hackathon)

5 What Works Today vs. Known Gaps

Verified end-to-end (safe to demo): flagship lost-NIC→passport flow with locked steps · eligibility Q&A pause/resume (both rules-based and generated questions) · blocked verdict with alternative path · step completion → live unlocking · custom plans for novel goals · case-relevant document checklists · document upload with Gemini Vision verdicts · dashboard with resumable cases · source citations. 90 backend tests green.

Known gaps — never volunteer, but answer honestly if asked (framing in PRESENT.md §6): identity is demo-mode (any email → signed JWT; the token contract matches what real NextAuth would issue) · reminders are computed but no SMS/WhatsApp delivery · only 4 procedures have fully curated rules (deliberate: depth over breadth) · custom plans carry no citations (they’re LLM knowledge, rules-verified

only for prerequisites) · deep Sinhala/Tamil localization is partial · the What-If simulation agent from the spec is designed but not built.

6 Future Development Plans

1. **Expand the verified dataset** — a procedure JSON per high-traffic service (marriage registration, business registration, citizenship, land transfer). Each new file instantly becomes a deterministic, cited workflow — no code changes.
2. **Production identity** — swap demo tokens for NextAuth (magic-link / Google); optional Postgres row-level security.
3. **Reminder delivery** — SMS / WhatsApp nudges for appointments, expiries, and idle cases.
4. **What-If simulation agent** — “Can I apply without my NIC?” → simulate alternative paths from the existing dependency graph.
5. **Government integrations** — appointment booking, payment, document verification APIs, and an officer-facing dashboard for Divisional Secretariats.
6. **Accessibility** — voice assistant in Sinhala/Tamil, kiosk mode for government offices, offline-tolerant flows for rural users.
7. **Analytics for policy makers** — where citizens get stuck, most-failed documents, broker-fraud warnings.

7 Meeting Prep — Who Says What

- **Demo driver:** rehearse the 4-act script in PRESENT.md §3 (flagship flow → eligibility pause → step unlock → custom goal). Run the backend *without* `--reload`; pre-seed 2–3 dashboard cases; warm up one run before the meeting.
- **Everyone:** memorize the one-liner, the “rules decide structure” quote, and the three closers in PRESENT.md §9.
- **Q&A owners:** assign the five standard questions (why not ChatGPT · what’s agentic · hallucinations · data provenance · business model) — full model answers in PRESENT.md §5.
- **Buyer story:** Government of Sri Lanka / ICTA. Value = fewer incomplete applications, less counter congestion, analytics, broker-fraud defense. Entire demo runs on free tiers — marginal cost per case is cents.

HelpLK AI · Agentic AI + RAG Hackathon · This document summarizes the system as actually built and verified. For the full presentation script, pre-flight checklist, and failure fallbacks see PRESENT.md in the repository root.